

FrontierAtlas Intelligence Graph — Architecture & Production Design

AI Engineer trial task · Scale, LLM orchestration, freshness, and storage strategy

1. Scale Strategy — Collecting 500,000+ Records Without Manual Intervention

The pipeline is built as a set of stateless, horizontally-scalable async workers rather than a single long-running script. Each vertical (startups, products, papers, jobs, news) is driven by a work-queue of discrete units — an Arxiv page offset, a directory pagination URL, a company slug — pushed onto a queue (Redis/SQS in production; an in-process `asyncio.Queue` in this trial). Any number of worker processes can pull from the same queue concurrently, so going from 1,000 to 500,000 records is purely an infrastructure change (spin up more workers / raise concurrency) and never a code change, satisfying the assignment's core scale constraint.

Within a worker, concurrency is bounded by an `asyncio.Semaphore` per downstream dependency (source site, GitHub API, LLM provider) so we saturate available throughput without tripping rate limits or overwhelming a single host. A token-bucket `RateLimiter` (see `src/scrapers/research_papers.py`) enforces per-source courtesy intervals independently of how many workers are running. Discovery (finding new URLs/IDs to crawl) and extraction (fetching + parsing a known URL) are decoupled into separate queues, so a slow discovery phase never blocks extraction throughput, and vice versa.

For the research-papers vertical specifically, we lean on Arxiv's and GitHub's official REST/Atom APIs rather than HTML scraping — this is both more reliable at scale and avoids anti-bot friction entirely for a source that already offers structured, bulk-friendly access.

2. Handling 413s and 429s Across Thousands of Concurrent Extractions

429 (rate limit): Every provider call goes through `LLMOrchestrator._call_with_backoff`, which retries with exponential backoff (base 1s, doubling, capped at 60s) plus randomized jitter ($\pm 50\%$) to avoid thundering-herd retries across concurrent workers hitting the same provider simultaneously. After `max_retries` is exhausted on a provider, the orchestrator does not keep hammering it — it falls through to the next tier in the fallback chain (Gemini Flash → OpenAI ChatGPT / GPT-4o) for the remainder of that document, since a provider that's rate-limited for one chunk is almost certainly rate-limited for the rest.

413 (payload too large): Before any provider call, raw text is chunked via `chunk_text()` to a budget derived from that provider's `max_context_tokens` (Gemini ~900k, OpenAI ChatGPT 128k), so the same document is chunked differently per tier automatically. Chunking prefers paragraph, then sentence boundaries to preserve semantic density; if a single chunk still triggers a 413 from a provider (context estimate was off, or provider-side limits are tighter than advertised), the orchestrator halves that specific chunk and retries up to twice before giving up on that chunk and moving to the next fallback tier. Chunks carry a small character overlap so entity/claim continuity isn't lost at chunk boundaries.

At the concurrency level, thousands of simultaneous extractions are bounded by a per-provider semaphore sized to that provider's published RPS/TPM limits, so we approach — but don't exceed — the rate ceiling, which keeps the 429 rate low in the first place rather than relying purely on retries to paper over it.

3. Freshness Tracking Across Distributed Crawler Nodes

Two layers guarantee both (a) no duplicate processing and (b) accurate 24-hour freshness:

- **Date normalization** (`src/utls/dates.py`) converts every source's publish date — ISO, relative ('2 hours ago'), or missing — into a canonical UTC ISO-8601 timestamp. Items are only accepted into the News/Jobs tabs if `is_within_last_24h()` returns true against a shared, synchronized clock.
- **Distributed dedupe state:** each item's dedupe key (canonical URL, or source+title hash where URLs are unstable) is checked against a shared, centralized store — Redis SET or a Postgres unique index in production — BEFORE any node commits it. This is the single source of truth across all worker nodes, so two nodes racing to crawl the same job

posting can't both insert it; the second write is rejected at the database layer via an idempotent UPSERT keyed on the dedupe key. The in-repo SeenBeforeHeuristic class is the local-file version of this same contract for the trial-scale run.

When a source gives no reliable date at all, we fall back to the seen-before heuristic: an item is treated as fresh only if its dedupe key was absent from the last run's persisted key set — never by assuming freshness by default, which would risk re-ingesting stale content as 'new.'

4. Storage Strategy

Primary store — PostgreSQL: the canonical schemas (Startup, Product, ResearchPaper, Job, News, Entity Mapping Log) are inherently relational — products belong to startups, papers link to repos, jobs link to companies — and need strong consistency guarantees (unique constraints for dedupe, transactional upserts). Postgres' native JSONB columns also let us store the full validated record payload alongside indexed relational columns (entity_id, source_url, collected_at), giving schema flexibility without giving up query performance or ACID guarantees, which a pure document store wouldn't.

Graph layer — Neo4j (or Postgres + Apache AGE for a lighter footprint): relationships like startup → product → paper → author → github_repo are naturally multi-hop graph queries ('show me all papers with code from startups founded by ex-DeepMind researchers'). Modeling this in a graph store avoids expensive recursive joins in Postgres as the relationship depth grows, and is where the 'Intelligence Graph' product concept — not just a records dump — actually gets realized.

Vector layer — pgvector or a dedicated vector DB (Pinecone/Weaviate): embeddings of paper abstracts, product descriptions, and news content power semantic search and entity-resolution fuzzy matching beyond what rapidfuzz string similarity alone can catch (e.g. two differently-worded descriptions of the same product). Starting with pgvector keeps the primary store's transactional guarantees for embeddings tied 1:1 to their source record; a dedicated vector DB is a later optimization once query volume justifies the operational cost.

Object storage — S3-compatible bucket: raw HTML/PDF snapshots are archived alongside the structured record they were extracted from, so every record is auditable back to its original source content, not just its source URL — directly supporting the assignment's 'no hallucinated data, every record traces to a valid source' requirement.